GLB/GLTF Character Plugin Integration Guide

Addendum to Freque Plugin Development Guide v2.3

Date: 2025-11-14
Topic: Procedural Skeletal Animation and Character Integration

Overview

This document covers the implementation of procedural skeletal animation systems for Three.js plugins that use GLB/GLTF character models. These patterns enable:

- **Skeletal bone manipulation** in Three.js without relying solely on pre-baked animations
- **Audio-reactive procedural animation** synced to audio frequencies
- **Multi-cycle rhythm layering** for organic, non-repetitive movement
- **Relative bone transformations** that preserve base animation poses
- **Mode-based animation systems** with proper state management

These techniques can be applied to any character animation plugin regardless of the specific movement style (dancing, walking, gesturing, etc.).

Table of Contents

Core Concepts

Procedural vs Pre-Baked Animations

Pre-baked Animations: - Stored in GLB/GLTF files as animation clips - Fixed sequences of bone rotations created in 3D software - Examples: Walk, Run, Jump, Idle, custom animations

Procedural Animations: - Calculated in real-time by code - Bone rotations computed each frame based on parameters - Can be audio-reactive and dynamic - Examples: Audio-reactive gestures, custom movement modes

Why Use Procedural Animation?

✅ **Audio-reactive** - Syncs perfectly to music in real-time
✅ **Dynamic** - Never repeats exactly the same way
✅ **Customizable** - User controls via UI sliders/toggles
✅ **Performant** - Simple calculations vs. complex animation blending
✅ **Flexible** - Can layer on top of or blend with pre-baked animations

---

Implementation Architecture

System Components

```
Procedural Animation System
│
├─ Base Rotation Capture
│  └─ Stores starting pose from pre-baked animation
│
├─ Multi-Cycle Rhythm Generator
│  ├─ Main beat (configurable BPM)
│  ├─ Slow phase (half-speed)
│  ├─ Long cycle (e.g., 30s)
│  ├─ Medium cycle (e.g., 12s)
│  └─ Short cycle (e.g., 7s)
│
├─ Bone Animation Layers
│  ├─ Root/Hips (minimal - affects lower body)
│  ├─ Spine/Torso (major movement source)
│  ├─ Head (bob, nod, sway)
│  ├─ Shoulders (lift, roll, shrug)
│  └─ Arms/Hands (swing, gesture, position)
│
└─ Audio Integration
   ├─ Bass → Beat intensity
   ├─ Mid → Variation timing
   └─ Treble → Movement accents
```

Animation State Machine

```
Audio Energy State Management:


┌─────────────────┐
│  No Audio       │ → Pre-baked Idle Animation
│ Energy < MIN    │
└─────────────────┘
        ↓
┌─────────────────┐
│ Custom Mode     │ → Procedural Animation
│ Energy ≥ MIN    │   (Mode ON)
│  & Mode ON      │
└─────────────────┘
        ↓
┌─────────────────┐
│ Default Anims   │ → Pre-baked Animations
│ Energy ≥ MIN    │   (Mode OFF)
│  & Mode OFF     │
└─────────────────┘
```

Critical Technical Patterns

1. Relative Bone Transformations

❌ WRONG: Absolute Rotation (Breaks Character Pose)

```
// This OVERWRITES the base animation pose
this.bones.hips.rotation.z = 0.1;  // Character appears incorrectly rotated!
```

✅ CORRECT: Relative to Base Pose

```
// Store base rotation from pre-baked animation
if (!this.baseRotations) {
    this.baseRotations = {};
    this.baseRotations.hips = {
        x: this.bones.hips.rotation.x,
        y: this.bones.hips.rotation.y,
        z: this.bones.hips.rotation.z
    };
}

// Apply procedural movement as OFFSET
const proceduralMovement = Math.sin(time) * 0.1;
this.bones.hips.rotation.z = this.baseRotations.hips.z + proceduralMovement;  // ✅
```

Why This Matters: - Pre-baked animations already position bones correctly - Your procedural movement adds to that, not replaces it - Character maintains proper orientation and pose - Switching between animations doesn’t break pose

2. Understanding Bone Hierarchy

Key Rule: Root/hip bones affect all child bones below them in the hierarchy.

```
Typical Character Skeleton Hierarchy:

Root/Hips
├─ Spine_Lower
│  └─ Spine_Upper
│     └─ Neck
│        └─ Head
│
├─ Left_UpperLeg
│  └─ Left_LowerLeg
│     └─ Left_Foot
│
└─ Right_UpperLeg
   └─ Right_LowerLeg
      └─ Right_Foot
```

Critical Insight: Rotating hips affects legs and feet. For stationary character movement:

```
// ✅ MINIMAL hip rotation (affects feet via hierarchy)
const hipAmount = 0.01;  // Very small - keeps feet stable

// ✅ MAJOR spine/torso rotation (creates visible movement WITHOUT moving feet)
const spineAmount = 0.25;  // Larger - visible sway without foot movement
```

3. Multi-Cycle Rhythm Layering

Single sine wave = repetitive, robotic:

```
const movement = Math.sin(time);  // Same pattern repeats every cycle
```

Multiple overlapping cycles = organic, alive:

```
// Multiple rhythm layers with different frequencies
const longCycle = time * 0.2 * Math.PI * 2;   // 30 second cycle
const mediumCycle = time * 0.5 * Math.PI * 2; // 12 second cycle
const shortCycle = time * 0.9 * Math.PI * 2;  // 7 second cycle

// Combine for variation
const variation1 = Math.sin(longCycle) * 0.5 + 0.5;  // 0-1 range

// Apply to movement amounts
const movementAmount = baseAmount * (0.7 + variation1 * 0.6);  // Varies 0.7x-1.3x
```

Why It Works: - Different cycle lengths create complex patterns - Movements never repeat exactly the same way - Feels like organic behavior with natural variation - Full pattern repeats after very long time (e.g., 90+ seconds)

4. Compound Movement Layers

Simple movement:

```
const headBob = Math.sin(beatPhase) * 0.08;
this.bones.head.rotation.x = baseRotation + headBob;
```

Compound movement (more natural):

```
// Primary movement on main beat
const headBob = Math.sin(beatPhase) * 0.08 * (0.5 + variation3);

// Secondary movement on long cycle
const headNod = Math.sin(longCycle * 0.5) * 0.03;

// Combine both layers
this.bones.head.rotation.x = baseRotation + headBob + headNod;
```

Result: Multiple movement frequencies = more organic, less mechanical.

Audio Reactivity

Energy Mapping Strategy

```
// Extract audio frequency bands
const bassEnergy = this.getEnergyInRange(audioData, 0, 150);     // 0-150 Hz
const midEnergy = this.getEnergyInRange(audioData, 150, 500);    // 150-500 Hz
const highEnergy = this.getEnergyInRange(audioData, 500, 2000);  // 500-2000 Hz

// Normalize to 0-1 range
const bassNorm = bassEnergy / 255;
const midNorm = midEnergy / 255;
const highNorm = highEnergy / 255;

// Calculate overall energy with sensitivity multiplier
const totalEnergy = ((bassNorm + midNorm + highNorm) / 3) * this.audioSensitivity;
```

Beat Detection for Character Animation

```
// Beat detection - use bass energy for rhythm
const beatIntensity = Math.min(bassEnergy * 2, 1.0);  // 0-1 range, clamped

// Apply to movement intensity
const movementAmount = baseAmount * (1 + beatIntensity * 0.5);
const movement = Math.sin(beatPhase) * movementAmount;

// Stronger beats = more pronounced movement
```

Audio-Driven State Transitions

```
const MIN_ENERGY_THRESHOLD = 0.05;

if (this.customMode && totalEnergy >= MIN_ENERGY_THRESHOLD) {
    // Custom Mode: ON + audio present → Apply procedural animation
    this.applyProceduralAnimation(deltaTime, bassNorm, midNorm, highNorm);

} else if (!this.customMode && totalEnergy > MIN_ENERGY_THRESHOLD) {
    // Custom Mode: OFF + audio present → Use pre-baked animations
    this.updateAnimationBasedOnEnergy(totalEnergy, bassNorm, midNorm, highNorm);

} else {
    // No audio → Return to idle
    this.switchToIdleAnimation();
}
```

UI Controls

Mode Toggle Control

Example: Custom Animation Mode Toggle

```
// In constructor:
this.customAnimationMode = true;  // Default to ON
this.baseRotations = null;  // Will be captured on first frame

// In setupControls():
this.addControl('customAnimationMode', {
    type: 'dropdown',
    label: 'Custom Animation',
    options: [
        { value: 'off', label: 'Off' },
        { value: 'on', label: 'On' }
    ],
    value: 'on',
    className: 'dropdown-selector-mixer',  // REQUIRED for styling
    onChange: (value) => {
        this.customAnimationMode = (value === 'on');
        // Clear base rotations so they're recaptured when mode changes
        this.baseRotations = null;
        console.log('Custom animation mode:', this.customAnimationMode ? 'ON' : 'OFF');
    }
});
```

Why Reset baseRotations? - Different animations have different base poses - Walking pose ≠ Idle pose ≠ Running pose - Must recapture bones from current animation when switching modes

Audio Sensitivity Control

```
this.addControl('audioSensitivity', {
    type: 'slider',
    label: 'Audio Sensitivity',
    min: 0.1,
    max: 3.0,
    step: 0.1,
    value: 1.0,
    onChange: (value) => {
        this.audioSensitivity = value;
    }
});
```

Effect: - **0.1**: Minimal audio response (subtle movement) - **1.0**: Normal audio response (default) - **3.0**: Exaggerated audio response (intense movement)

Character Position Controls

```
// X position (horizontal)
this.addControl('characterX', {
    type: 'slider',
    label: 'X Position',
    min: -2,
    max: 2,
    step: 0.1,
    value: 0,
    onChange: (value) => {
        this.characterX = value;
        if (this.character) {
            this.character.position.x = value;
        }
    }
});

// Y position (vertical)
this.addControl('characterY', {
    type: 'slider',
    label: 'Y Position',
    min: -1,
    max: 1,
    step: 0.1,
    value: 0.4,
    onChange: (value) => {
        this.characterY = value;
        if (this.character) {
            this.character.position.y = value;
        }
        // Note: Don't call updateCameraPosition() here if you want
        // character to move independently of camera
    }
});
```

Common Pitfalls & Solutions

Problem 1: Character Rotates Incorrectly in Custom Mode

Symptom: Character orientation changes when custom animation mode activates

❌ **WRONG Approach:**

```
// Rotating the entire character to "fix" orientation
this.character.rotation.y = -Math.PI / 2;  // DON'T DO THIS
```

✅ **CORRECT Approach:** The issue is **absolute bone rotations overwriting base pose**:

```
// Store base rotation FIRST (from current pre-baked animation)
if (!this.baseRotations) {
    this.baseRotations = {
        hips: {
            x: this.bones.hips.rotation.x,
            y: this.bones.hips.rotation.y,
            z: this.bones.hips.rotation.z
        }
        // ... other bones
    };
}

// Apply procedural animation as OFFSET to base
this.bones.hips.rotation.z = this.baseRotations.hips.z + proceduralOffset;
```

Problem 2: Character's Feet Move/Float/Slide

Symptom: Character's feet leave the ground or slide during procedural animation

Cause: Too much root/hip rotation affects legs via bone hierarchy

✅ **Solution:**

```
// Reduce hip movement drastically (root of leg hierarchy)
const hipAmount = 0.01 + (beatIntensity * 0.005);  // 0.01-0.015 radians MAX

// Increase spine/torso movement instead (above legs in hierarchy)
const spineAmount = 0.25 * (1 + beatIntensity * 0.5);  // Up to ~0.38 radians
```

Rule of Thumb: Hip rotation < 0.02 radians keeps feet stable for stationary characters.

Problem 3: Animation Feels Repetitive/Robotic

Symptom: Same movements repeat obviously, feels mechanical

❌ **WRONG Approach:**

```
// Single sine wave - repeats every cycle
const movement = Math.sin(time * speed);
```

✅ **CORRECT Approach:**

```
// Multiple overlapping cycles
const longCycle = time * 0.2 * Math.PI * 2;   // 30s cycle
const mediumCycle = time * 0.5 * Math.PI * 2; // 12s cycle
const shortCycle = time * 0.9 * Math.PI * 2;  // 7s cycle

const variation = Math.sin(longCycle) * 0.5 + 0.5;  // 0-1 modulator
const movement = Math.sin(time * speed) * baseAmount * (0.7 + variation * 0.6);
```

Problem 4: Animation Too Fast/Slow for Music

Symptom: Movement tempo doesn’t match music feel

✅ **Solution - Adjust Main Rhythm Speed:**

```
// Slower rhythm (36 BPM)
const mainSpeed = 0.6;

// Normal rhythm (60 BPM)
const mainSpeed = 1.0;

// Faster rhythm (90 BPM)
const mainSpeed = 1.5;

const beatPhase = time * mainSpeed * Math.PI * 2;
```

Problem 5: Animation Continues When Mode Turned Off

Symptom: Procedural movements persist after toggling mode off

✅ **Solution - Proper State Management:**

```
// Only apply procedural animation when mode is enabled AND audio is present
const MIN_ENERGY = 0.05;

if (this.customMode && totalEnergy >= MIN_ENERGY) {
    this.applyProceduralAnimation(deltaTime, bassNorm, midNorm, highNorm);

} else if (!this.customMode && totalEnergy > MIN_ENERGY) {
    // Use pre-baked animations instead
    this.updatePrebakedAnimations(totalEnergy, bassNorm, midNorm, highNorm);

} else {
    // Return to idle
    this.switchToIdleAnimation();
}
```

Code Examples

Complete Procedural Animation Method Structure

```
/**
 * Apply procedural animation to character bones
 * @param {number} deltaTime - Time since last frame
 * @param {number} bassEnergy - Normalized bass energy (0-1)
 * @param {number} midEnergy - Normalized mid energy (0-1)
 * @param {number} highEnergy - Normalized high energy (0-1)
 */
applyProceduralAnimation(deltaTime, bassEnergy, midEnergy, highEnergy) {
    if (!this.bones || !this.bones.hips || !this.customMode) return;

    // ═══════════════════════════════════════════════════════════
    // STEP 1: Store base rotations on first frame
    // ═══════════════════════════════════════════════════════════
    if (!this.baseRotations) {
        this.baseRotations = {};

        // Store hips
        if (this.bones.hips) {
            this.baseRotations.hips = {
                x: this.bones.hips.rotation.x,
                y: this.bones.hips.rotation.y,
                z: this.bones.hips.rotation.z
            };
        }

        // Store spine bones (array)
        if (this.bones.spine && this.bones.spine.length > 0) {
            this.baseRotations.spine = this.bones.spine.map(bone => ({
                x: bone.rotation.x,
                y: bone.rotation.y,
                z: bone.rotation.z
            }));
        }

        // Store other bones as needed (head, shoulders, arms, etc.)
```

```
        // ... similar pattern for each bone
    }

    // ═══════════════════════════════════════════════════════════
    // STEP 2: Update animation time
    // ═══════════════════════════════════════════════════════════
    this.animationTime += deltaTime;

    // ═══════════════════════════════════════════════════════════
    // STEP 3: Calculate beat intensity from audio
    // ═══════════════════════════════════════════════════════════
    const beatIntensity = Math.min(bassEnergy * 2, 1.0); // 0-1 range

    // ═══════════════════════════════════════════════════════════
    // STEP 4: Setup rhythm cycles
    // ═══════════════════════════════════════════════════════════
    const mainSpeed = 0.6; // Adjust for desired BPM
    const beatPhase = this.animationTime * mainSpeed * Math.PI * 2;
    const slowPhase = beatPhase * 0.5;

    // Additional longer cycles for variation
    const longCycle = this.animationTime * 0.2 * Math.PI * 2;   // 30s
    const mediumCycle = this.animationTime * 0.5 * Math.PI * 2; // 12s
    const shortCycle = this.animationTime * 0.9 * Math.PI * 2;  // 7s

    // ═══════════════════════════════════════════════════════════
    // STEP 5: Create variation multipliers (0-1 range)
    // ═══════════════════════════════════════════════════════════
    const variation1 = Math.sin(longCycle) * 0.5 + 0.5;
    const variation2 = Math.sin(mediumCycle) * 0.5 + 0.5;
    const variation3 = Math.sin(shortCycle) * 0.5 + 0.5;

    // ═══════════════════════════════════════════════════════════
    // STEP 6: Apply to HIPS (minimal - affects lower body)
    // ═══════════════════════════════════════════════════════════
    if (this.bones.hips && this.baseRotations.hips) {
```

```
        // Z-axis (side-to-side sway)
        const hipSwayAmount = (0.01 + (beatIntensity * 0.005)) * (0.7 + variation1 * 0.6)
        const hipSway = Math.sin(slowPhase + longCycle * 0.3) * hipSwayAmount;
        this.bones.hips.rotation.z = this.baseRotations.hips.z + hipSway;

        // X-axis (forward/back rock)
        const hipRockAmount = (0.005 + (beatIntensity * 0.003)) * (0.8 + variation2 * 0.4
        const hipRock = Math.sin(beatPhase + mediumCycle * 0.2) * hipRockAmount;
        this.bones.hips.rotation.x = this.baseRotations.hips.x + hipRock;
    }

    // ═══════════════════════════════════════════════════════════
    // STEP 7: Apply to SPINE/TORSO (major movement source)
    // ═══════════════════════════════════════════════════════════
    if (this.bones.spine && this.bones.spine.length > 0 && this.baseRotations.spine) {
        // Primary twist with audio reactivity
        const spineTwist = Math.sin(slowPhase + Math.PI) * 0.25 * (1 + beatIntensity * 0.
        // Secondary motion layer
        const spineSecondary = Math.sin(mediumCycle) * 0.08;
        this.bones.spine[0].rotation.y = this.baseRotations.spine[0].y + spineTwist + spi

        // Upper spine (if exists) - amplified movement
        if (this.bones.spine[1]) {
            const upperTwist = (spineTwist * 0.9) * (0.7 + variation2 * 0.6);
            this.bones.spine[1].rotation.y = this.baseRotations.spine[1].y + upperTwist;
        }
    }

    // ═══════════════════════════════════════════════════════════
    // STEP 8: Apply to HEAD
    // ═══════════════════════════════════════════════════════════
    if (this.bones.head && this.baseRotations.head) {
        // X-axis (bob/nod)
        const headBobAmount = (0.08 + (beatIntensity * 0.05)) * (0.5 + variation3 * 1.0);
        const headBob = Math.sin(beatPhase) * headBobAmount;
        const headNod = Math.sin(longCycle * 0.5) * 0.03;
```

```
        this.bones.head.rotation.x = this.baseRotations.head.x + headBob + headNod;

        // Z-axis (side-to-side sway)
        const headSwayAmount = (0.06 + (beatIntensity * 0.03)) * (0.6 + variation1 * 0.8)
        const headSway = Math.sin(slowPhase + shortCycle * 0.3) * headSwayAmount;
        this.bones.head.rotation.z = this.baseRotations.head.z + headSway;
    }

    // ═══════════════════════════════════════════════════════════
    // STEP 9: Apply to SHOULDERS (alternating)
    // ═══════════════════════════════════════════════════════════
    if (this.bones.rightArm && this.baseRotations.rightArm) {
        const rightLift = Math.sin(beatPhase) * (0.15 + beatIntensity * 0.08) * (0.5 + va
        const rightRoll = Math.sin(mediumCycle) * 0.05;
        this.bones.rightArm.rotation.z = this.baseRotations.rightArm.z + rightLift + righ
    }

    if (this.bones.leftArm && this.baseRotations.leftArm) {
        // Opposite phase for alternating movement
        const leftLift = Math.sin(beatPhase + Math.PI) * (0.15 + beatIntensity * 0.08) *
        const leftRoll = Math.sin(mediumCycle + Math.PI) * 0.05;
        this.bones.leftArm.rotation.z = this.baseRotations.leftArm.z + leftLift + leftRol
    }

    // ═══════════════════════════════════════════════════════════
    // STEP 10: Apply to ARMS/FOREARMS (compound swing)
    // ═══════════════════════════════════════════════════════════
    if (this.bones.rightForearm && this.baseRotations.rightForearm) {
        const rightSwing = Math.sin(slowPhase) * (0.2 + beatIntensity * 0.15) * (0.6 + va
        const rightAccent = Math.sin(shortCycle) * 0.08;
        this.bones.rightForearm.rotation.x = this.baseRotations.rightForearm.x + rightSwi
    }

    if (this.bones.leftForearm && this.baseRotations.leftForearm) {
        const leftSwing = Math.sin(slowPhase + Math.PI) * (0.2 + beatIntensity * 0.15) *
        const leftAccent = Math.sin(shortCycle + Math.PI * 0.7) * 0.08;
```

```
        this.bones.leftForearm.rotation.x = this.baseRotations.leftForearm.x + leftSwing
    }

    // ═══════════════════════════════════════════════════════════════
    // STEP 11: Update skeleton (required for skinned meshes)
    // ═══════════════════════════════════════════════════════════════
    if (this.skeleton) {
        this.skeleton.update();
    }
}
```

Integration in onUpdate Method

```
onUpdate(deltaTime, timestamp, sharedAudioData) {
    if (!this.character || !this.mixer) return;

    // Clamp deltaTime to prevent animation jumps
    const clampedDelta = Math.max(0.001, Math.min(deltaTime, 0.05));

    // Update animation mixer (for pre-baked animations)
    this.mixer.update(clampedDelta);

    // ═══════════════════════════════════════════════════════════
    // Audio-reactive logic
    // ═══════════════════════════════════════════════════════════
    if (sharedAudioData && sharedAudioData.dataArray) {
        // Extract frequency bands
        const bassEnergy = this.getEnergyInRange(sharedAudioData.dataArray, 0, 150);
        const midEnergy = this.getEnergyInRange(sharedAudioData.dataArray, 150, 500);
        const highEnergy = this.getEnergyInRange(sharedAudioData.dataArray, 500, 2000);

        // Normalize to 0-1
        const bassNorm = bassEnergy / 255;
        const midNorm = midEnergy / 255;
        const highNorm = highEnergy / 255;

        // Calculate total energy with sensitivity
        const totalEnergy = ((bassNorm + midNorm + highNorm) / 3) * this.audioSensitivity

        const MIN_ENERGY = 0.05;

        // ═══════════════════════════════════════════════════════
        // State management: Custom mode vs Pre-baked animations
        // ═══════════════════════════════════════════════════════
        if (this.customMode && totalEnergy >= MIN_ENERGY) {
            // CUSTOM MODE ON: Apply procedural animation
            this.applyProceduralAnimation(clampedDelta, bassNorm, midNorm, highNorm);
```

```
            // Ensure Idle animation is playing as base pose
            const currentAnimName = this.currentAction?.getClip().name;
            if (currentAnimName !== 'Idle') {
                this.switchToAnimation('Idle');
            }

        } else if (!this.customMode && totalEnergy > MIN_ENERGY) {
            // CUSTOM MODE OFF: Use pre-baked animations based on energy
            this.updatePrebakedAnimations(totalEnergy, bassNorm, midNorm, highNorm);

        } else {
            // NO AUDIO: Return to Idle
            if (this.currentAction?.getClip().name !== 'Idle') {
                this.switchToAnimation('Idle');
            }
        }
    }
}
```

Extracting Bones from GLB/GLTF

```
/**
 * Extract skeleton and specific bones for procedural animation
 */
extractBones() {
    if (!this.character) return;

    // Find SkinnedMesh
    let skinnedMesh = null;
    this.character.traverse((child) => {
        if (child.isSkinnedMesh && child.skeleton) {
            skinnedMesh = child;
        }
    });

    if (!skinnedMesh || !skinnedMesh.skeleton) {
        console.warn('No skeleton found in character model');
        return;
    }

    this.skeleton = skinnedMesh.skeleton;
    console.log(`Found skeleton with ${this.skeleton.bones.length} bones`);

    // Create bone map for easy access
    this.bones = {};

    // Find specific bones by name (adjust names to match your model)
    this.skeleton.bones.forEach(bone => {
        const name = bone.name.toLowerCase();

        // Hips/Root
        if (name.includes('hips') || name.includes('pelvis') || name.includes('root')) {
            this.bones.hips = bone;
        }

        // Spine (may have multiple segments)
```

```
        if (name.includes('spine')) {
            if (!this.bones.spine) this.bones.spine = [];
            this.bones.spine.push(bone);
        }

        // Head
        if (name.includes('head')) {
            this.bones.head = bone;
        }

        // Right arm/shoulder
        if (name.includes('right') && (name.includes('arm') || name.includes('shoulder'))
            if (name.includes('upper') || name.includes('shoulder')) {
                this.bones.rightArm = bone;
            } else if (name.includes('lower') || name.includes('forearm')) {
                this.bones.rightForearm = bone;
            }
        }

        // Left arm/shoulder
        if (name.includes('left') && (name.includes('arm') || name.includes('shoulder')))
            if (name.includes('upper') || name.includes('shoulder')) {
                this.bones.leftArm = bone;
            } else if (name.includes('lower') || name.includes('forearm')) {
                this.bones.leftForearm = bone;
            }
        }
    });

    // Log found bones for debugging
    console.log('Extracted bones:', {
        hips: !!this.bones.hips,
        spine: this.bones.spine?.length || 0,
        head: !!this.bones.head,
        rightArm: !!this.bones.rightArm,
        rightForearm: !!this.bones.rightForearm,
```

```
        leftArm: !!this.bones.leftArm,
        leftForearm: !!this.bones.leftForearm
    });
}
```

Best Practices for Procedural Skeletal Animation

1. Always Use Relative Transformations

```
// ✅ Store base, apply offsets
const baseRot = this.bones.hips.rotation.z;
const offset = Math.sin(time) * 0.1;
this.bones.hips.rotation.z = baseRot + offset;
```

2. Minimize Root Bone Movement

```
// ✅ Small hip rotation (keeps feet planted for stationary characters)
const hipAmount = 0.01;

// ✅ Large torso rotation (creates visible movement)
const spineAmount = 0.25;
```

3. Layer Multiple Frequencies

```
// ✅ Combine fast + slow rhythms
const primary = Math.sin(time * 2);      // Fast
const secondary = Math.sin(time * 0.3);  // Slow
const combined = primary + secondary * 0.5;
```

4. Add Variation to Avoid Repetition

```
// ✅ Modulate movement amounts over time
const variation = Math.sin(longCycle) * 0.5 + 0.5;
const amount = baseAmount * (0.7 + variation * 0.6);
```

5. Sync to Audio Meaningfully

```
// ✅ Bass → rhythm timing
const beatPhase = time * (bassEnergy + 0.5);

// ✅ Beat intensity → movement strength
const movementScale = 1.0 + beatIntensity * 0.3;
```

6. Clear State on Mode Changes

```
// ✅ Reset base rotations when switching modes
onChange: (value) => {
    this.customMode = (value === 'on');
    this.baseRotations = null;  // Recapture from new animation
}
```

Performance Considerations

Computational Cost

Per Frame: - 6-8 bone rotations (hips, spine × 2, head, shoulders × 2, arms × 2) - ~20 sine/cosine calculations - 3 variation calculations - **Total:** ~50-60 simple math operations

Impact: Negligible (<0.1ms per frame on modern hardware)

Memory Usage

- `baseRotations` object: ~200-500 bytes (depending on bone count)
- `animationTime` scalar: 8 bytes
- Temporary calculation variables: ~100 bytes

Total: <1KB additional memory

Optimization Tips

✅ Pre-calculate constants:

```
// In constructor or once per session
this.PI_OVER_2 = Math.PI / 2;
this.TWO_PI = Math.PI * 2;
```

✅ Reuse calculation results:

```
const sinSlowPhase = Math.sin(slowPhase);
const hipSway = sinSlowPhase * hipAmount;
const headSway = sinSlowPhase * headAmount;  // Reuse sine
```

✅ Conditional updates:

```
// Only update when custom mode is active
if (this.customMode && totalEnergy >= MIN_ENERGY) {
    this.applyProceduralAnimation(...);
}
```

Testing Checklist

When implementing procedural skeletal animation:

- ☐

Orientation: Character faces correct direction in all modes

- ☐ **Feet/Stability**: Character's feet stay planted (if intended)
- ☐ **Smoothness**: No jerky movements or discontinuities
- ☐ **Audio Sync**: Movement timing matches music beat
- ☐ **Variation**: Movement doesn't repeat obviously
- ☐ **Toggle**: Custom mode can be turned on/off cleanly
- ☐ **Base Pose**: Switching animations doesn't break character pose
- ☐ **Performance**: Maintains 60fps on target hardware
- ☐ **Bone Names**: Code correctly identifies bones from your specific model
- ☐ **Fallbacks**: System handles missing bones gracefully

Prompting Claude for GLB/GLTF Character Plugins

Example Prompt for Character Plugin

```
Claude, I need to create a Freque plugin that loads and animates a GLB/GLTF character:

REQUIREMENTS:
1. Load GLB/GLTF model from specified path
2. Setup Three.js scene with proper lighting
3. Support pre-baked animations from the GLB file (Idle, Walk, Run, etc.)
4. Add procedural skeletal animation mode with audio reactivity
5. Apply bone rotations RELATIVE to base animation pose (not absolute)
6. Layer multiple rhythm cycles to avoid repetition
7. Sync to bass frequency for beat detection
8. Toggle-able mode with proper state management

CONTROLS:
- Character Selector: dropdown to choose different GLB files
- Animation Mode: dropdown (Pre-baked / Procedural / Mixed)
- Audio Sensitivity: slider (0.1-3.0)
- Character Position: X and Y sliders
- Camera Distance: slider

AUDIO MAPPING:
- Bass → beat intensity (0-1) for procedural animation
- Overall energy → animation speed for pre-baked animations
- Procedural animation uses multi-cycle rhythm layers

MOVEMENT STYLE:
[Describe the type of movement: dancing, gesturing, idle motion, etc.]
[Specify: upper body focused, full body, minimal lower body, etc.]

Please implement following GLB/GLTF character integration patterns with proper
base rotation capture and multi-cycle layering.
```

Example Prompt for Mode-Specific Animation

```
Claude, add a custom procedural animation mode to an existing character plugin:

CONTEXT:
- Plugin already loads GLB/GLTF and plays pre-baked animations
- Need to add audio-reactive procedural bone manipulation
- Should work as a toggle (on/off) without breaking existing functionality

MOVEMENT TYPE:
[Describe: e.g., "slow swaying motion", "head bobbing and shoulder movement",
"gestural hand movements", etc.]

TECHNICAL REQUIREMENTS:
1. Store base bone rotations from current animation
2. Apply procedural movements as OFFSETS (not absolute values)
3. Minimal hip movement if character should stay stationary
4. Major torso/spine movement for visible effect
5. Multi-cycle rhythm (30s, 12s, 7s cycles) for variation
6. Beat detection from bass frequencies
7. Clear base rotations when toggling mode on/off

CONTROLS:
- [Mode Name] Mode: dropdown toggle (on/off)
- Audio Sensitivity: slider to control intensity

Please implement with proper relative transformations and state management.
```

Summary

Procedural skeletal animation in GLB/GLTF character plugins:

✅ **Use relative transformations** - Store base pose from pre-baked animations, apply procedural offsets

✅ **Understand bone hierarchy** - Root bones affect all children (hips → legs → feet)
✅ **Layer multiple frequencies** - Combine multiple cycles for organic variation
✅ **Sync to audio meaningfully** - Bass for beat detection, energy for intensity
✅ **Manage state properly** - Clear base rotations on mode/animation changes
✅ **Test thoroughly** - Orientation, smoothness, audio sync, performance
✅ **Handle bone names** - Different models use different naming conventions

These patterns can be applied to any GLB/GLTF character animation system for audio-reactive procedural movement in Freque plugins.

End of Document